

Lab 3 — Service Setup

HTTP · FTP · SSH · MySQL

Table of contents

1	Introduction	1
2	HTTP Server	2
2.1	Setup Steps	3
3	FTP Server	4
3.1	Setup Steps	5
4	SSH Server	7
4.1	Setup Steps	8
5	MySQL Server	10
5.1	Setup Steps	10
6	Summary	13

1 Introduction

In this lab you will install and configure four of the most common network services found in real-world Linux environments:

Service	Default Port	Protocol	Purpose
HTTP (Apache)	80	TCP	Serve web pages
FTP (vsftpd)	21	TCP	Transfer files
SSH (OpenSSH)	22	TCP	Secure remote shell
MySQL	3306	TCP	Relational database

Knowing how these services work—and how they can be misconfigured—is a core skill in cybersecurity. Attackers routinely scan for exactly these ports; by setting them up yourself you will understand both the attacker’s perspective and what a defender needs to protect.

💡 Checking Open Ports

Throughout this lab you will use the following command to verify that a service is listening on the expected port. Run it after each installation step.

```
sudo ss -tlnp
```

i A Note on `netstat`

You may see `netstat` used in older tutorials and documentation to check listening ports. It comes from the legacy `net-tools` package, which is increasingly not installed by default. The modern replacement is `ss` (from `iproute2`, the same package that provides `ip`), which is installed by default on current systems.

If you need `netstat` on an older or unfamiliar system:

```
sudo apt-get install net-tools
sudo netstat -alpn | grep ' LISTEN '
```

i A Note on `service`

This lab uses `systemctl restart <name>`, the native `systemd` command, to restart services. You may also see `sudo service <name> restart` in older tutorials — on Ubuntu and Mint this still works today as a compatibility wrapper around `systemctl`, so either will work, but `systemctl` is the current standard.

2 HTTP Server

i Background: Hypertext Transfer Protocol (HTTP)

HTTP (Hypertext Transfer Protocol) is the foundational communication protocol of the web, used to transfer data between clients (typically web browsers) and servers. It was developed in the early 1990s by Tim Berners-Lee as part of the creation of the World Wide Web at CERN. The original version supported only simple text retrieval, but it

quickly evolved into HTTP/1.1, HTTP/2, and HTTP/3, which add features like persistent connections, header compression, and multiplexed streams for better performance. HTTP is a **stateless** protocol — each request is independent. Websites that need to remember who you are (shopping carts, logged-in accounts) use add-ons like cookies or server-side sessions to work around this.

Why it matters in cybersecurity: HTTP is one of the most common attack surfaces on the internet. Vulnerabilities like cross-site scripting (XSS), SQL injection, and directory traversal all exploit weaknesses in how HTTP is handled. For this reason, HTTP is almost always paired with **HTTPS** (HTTP + TLS encryption) in production so that traffic cannot be read or tampered with in transit.

! Security Note: HTTP vs. HTTPS

Plain HTTP traffic is unencrypted — anyone on the same network can read it with a tool like Wireshark. In this lab we use plain HTTP for simplicity, but in any real deployment you would configure a TLS certificate (e.g., via Let's Encrypt) and redirect all traffic to port 443 (HTTPS).

Our HTTP server will be hosted on **port 80** (the default HTTP port). We will use **Apache2**, one of the most widely deployed web servers in the world.

2.1 Setup Steps

1. Install Apache.

```
sudo apt-get install apache2
```

2. Confirm that Apache is listening on port 80.

```
sudo ss -tlnp
```

Look for a line like the following, with port 80 and **apache2** in the process column:

```
LISTEN 0      511          *:80         *:.*        users:(("apache2",pid=29022,fd=4))
```

3. Visit the default website by opening a browser and navigating to <http://localhost>.

You should see the Apache2 Ubuntu default page. This page is served from a file on disk — which you will inspect in the next step.

4. Locate the web root. The HTML file powering that page is at `/var/www/html/index.html`. View it with:

```
sudo more /var/www/html/index.html
```

Any file you place in `/var/www/html/` will be publicly served by Apache. Try creating a simple `hello.html` file there and visiting <http://localhost/hello.html> to see it live.

3 FTP Server

i Background: File Transfer Protocol (FTP)

FTP (File Transfer Protocol) is one of the oldest network protocols, first standardized in the early 1970s and later formalized in [RFC 959](#). It was designed in an era when networks were largely trusted, so it has **no built-in encryption** — usernames, passwords, and file contents are all sent in plaintext.

FTP uses **two separate TCP connections**:

- A **control channel** (port 21) for commands like `LIST`, `RETR`, and `STOR`.
- A **data channel** (port 20 in active mode, or a negotiated port in passive mode) for the actual file transfer.

Active vs. Passive mode is a common source of confusion with firewalls. In *active* mode the server initiates the data connection back to the client, which firewalls often block. In *passive* mode the client initiates both connections, which is friendlier to NAT and firewalls.

Why it matters in cybersecurity: FTP is a classic example of a protocol that became a security liability over time. Because credentials travel in plaintext, a simple packet capture with Wireshark can reveal passwords. Secure alternatives are **SFTP** (FTP-like commands tunneled over SSH) and **FTPS** (FTP with TLS). Despite being legacy, FTP still appears frequently in penetration testing engagements and CTF challenges, often with anonymous login enabled — exactly what you will configure below.

! Security Note: Anonymous FTP

Enabling anonymous FTP (as you will do below) intentionally allows **anyone** to connect without a password. This is useful for public file distribution, but it is dangerous if misconfigured — for example, if write access is accidentally granted. Always make the anonymous root directory and its contents **read-only**.

Our FTP server will be hosted on **port 21** (the default FTP port). We will use **vsftpd** (Very Secure FTP Daemon), the default FTP server on Ubuntu.

3.1 Setup Steps

1. Install vsftpd.

```
sudo apt-get install vsftpd
```

2. Confirm that vsftpd is listening on port 21.

```
sudo ss -tlnp
```

Look for:

```
LISTEN  0      32          0.0.0.0:21          0.0.0.0:*    users:(("vsftpd",pid=3309,fd=
```

3. Log in with your VM user credentials to confirm basic FTP works.

```
ftp localhost
```

Enter your username and password when prompted. A successful login looks like:

```
Login successful.
```

```
Remote system type is UNIX.
```

```
Using binary mode to transfer files.
```

```
ftp>
```

4. List files in your home directory with:

```
ftp> ls -lh
```

5. Disconnect from the FTP session with Ctrl+D.

6. Open the vsftpd configuration file.

```
sudo nano /etc/vsftpd.conf
```

Troubleshooting Tip

If the file appears blank, close it and check that you typed the path exactly as shown. A blank file usually means the path was mistyped and nano created an empty new file instead.

7. **Edit the configuration** to enable anonymous access. Find and modify the variables below (use Ctrl+W in nano to search). If a line is commented out with #, remove the # to uncomment it.

```

anonymous_enable=YES
local_enable=NO

# Uncomment this line and set your own banner message
ftpd_banner=Welcome to the Lab FTP Server

# Add this line at the bottom if it does not already exist
# (the directory /home/anonymous/ftp does not exist yet
# - you will create it next)
anon_root=/home/anonymous/ftp

```

Variable	Purpose
anonymous_enable=YES	Allows login without a password
local_enable=NO	Prevents local system users from logging in
ftpd_banner	Custom welcome message shown on connect
anon_root	Directory shown to anonymous users

8. **Save and close** the config file (Ctrl+X, then Y, then Enter).
9. **Restart vsftpd** to apply the changes.

```
sudo systemctl restart vsftpd
```

10. **Create the anonymous root directory** and add a test file.

```

cd /home
sudo mkdir anonymous
cd anonymous
sudo mkdir ftp
cd ftp
sudo touch ftp-test

```

11. **Test anonymous FTP login.**

```
ftp localhost
```

- **Username:** anonymous
- **Password:** *(leave blank, just press Enter)*

Once logged in, run `ls -lh` and confirm that `ftp-test` is visible.

12. **Test the connection from your host machine.**

- a. Find your VM's IP address:

```
ip addr show
```

- b. From a terminal on your **host machine**, connect using the VM's IP:

```
ftp <VM-IP-ADDRESS>
```

- c. Run `ls` to verify you can see `ftp-test`.

💡 Windows Host

The `ftp` command works by default in PowerShell and Command Prompt.

💡 macOS Host

You may need to install the FTP client first:

```
brew install inetutils
```

If you do not have Homebrew, install it from <https://brew.sh>.

4 SSH Server

i Background: Secure Shell (SSH)

SSH (Secure Shell) is a cryptographic network protocol for securely accessing and managing remote systems. It was created in 1995 by Tatu Ylönen in response to a password-sniffing attack at his university — at the time, most remote login was done with **Telnet**, which sends everything in plaintext. SSH encrypts the entire session, protecting both commands and data from eavesdropping.

SSH supports two main authentication methods:

- **Password authentication** — familiar but susceptible to brute-force attacks.
- **Public-key authentication** — the client holds a private key; the server holds the matching public key. This is far more secure and is the recommended method for any production system.

Why it matters in cybersecurity: SSH is the most common way administrators manage Linux servers, and attackers know it. Port 22 is among the most heavily scanned ports on the internet. Common attack techniques include brute-force credential stuffing and exploiting misconfigured key permissions. Best practices include: disabling root login

(`PermitRootLogin no`), enforcing key-based auth only (`PasswordAuthentication no`), and using a tool like **fail2ban** to automatically block IPs after repeated failed logins.

! Security Note: Disable Root SSH Login

Allowing `root` to log in directly over SSH is a significant risk. If an attacker guesses or steals the root password, they have full control of the system. Always set `PermitRootLogin no` in `/etc/ssh/sshd_config` on any machine exposed to the internet.

Our SSH server will run on **port 22** (the default SSH port). We will use **OpenSSH**, the standard SSH implementation on Linux.

4.1 Setup Steps

1. **Install OpenSSH Server.**

```
sudo apt-get install openssh-server
```

2. **Confirm that `sshd` is listening on port 22.**

```
sudo ss -tlnp
```

Look for a line like the following:

```
LISTEN 0      128          0.0.0.0:22          0.0.0.0:*        users:((("sshd",pid=1091,fd=3)
```

3. **Navigate away from your home directory** so you can see the prompt change when SSH brings you back. For example:

```
cd /tmp
```

4. **Log in to the local SSH server.** The first time you connect, SSH will ask you to verify the server's fingerprint — type **yes** to accept and continue. Then enter your password.

```
ssh localhost
```

Notice that your prompt has changed back to your home directory — SSH always starts a session in the user's home directory regardless of where you called it from.

5. **Verify your location.**

```
pwd
```

6. **Disconnect** with `Ctrl+D`. You will see a message like `Connection to localhost closed`.

7. Log in from your host machine.

- a. Find your VM's IP address:

```
ip addr show
```

- b. On your **host machine**, open a terminal and run:

```
ssh <username>@<VM-IP-ADDRESS>
```

Replace `<username>` with your VM username and `<VM-IP-ADDRESS>` with the address from the previous step.

- c. Run `pwd` to confirm you are connected, then disconnect with `Ctrl+D`.

8. Configure SSH for lab/competition use.

- a. Open the SSH daemon configuration file:

```
sudo nano /etc/ssh/sshd_config
```

- b. Find the `UseDNS` line and ensure it reads:

```
UseDNS no
```

- c. Save, close the file, and restart SSH:

```
sudo systemctl restart ssh
```

i Why `UseDNS no`?

When `UseDNS yes` (the default), the SSH server performs a reverse DNS lookup on each connecting client. In a lab or competition environment with no DNS server configured, this lookup times out and causes a noticeable delay (often 5–30 seconds) before the login prompt appears. Setting it to `no` eliminates that delay.

5 MySQL Server

Background: MySQL

MySQL is an open-source relational database management system (RDBMS) that stores and retrieves data using **SQL (Structured Query Language)**. It was originally developed in the mid-1990s — the “My” in its name comes from the daughter of co-founder Michael Widenius, and “SQL” is the query language it uses. MySQL is a cornerstone of the **LAMP stack** (Linux, Apache, MySQL, PHP/Python/Perl) that powers a large portion of the internet.

MySQL follows a client-server model: applications send SQL queries to the MySQL server, which processes them and returns results. Data is organized into **databases** containing **tables**, each with rows and columns.

Why it matters in cybersecurity: Databases are prime targets because they store the most sensitive data — passwords, personal information, financial records. Key attack techniques include:

- **SQL Injection** — crafting malicious input to manipulate queries and dump data.
- **Privilege escalation** — abusing overly permissive user grants.
- **Weak credentials** — default or blank passwords on the root account.

Defenses include prepared statements (to prevent injection), the principle of least privilege (users get only the access they need), regular patching, and binding MySQL to localhost unless remote access is genuinely required.

Security Note: Binding MySQL to 0.0.0.0

In step 6 below you will change `bind-address` from `127.0.0.1` to `0.0.0.0`, allowing remote connections. On a real server, this should be paired with strict firewall rules (e.g., `ufw allow from <trusted-IP> to any port 3306`) so that the database is not exposed to the entire internet.

Your MySQL server will run on **port 3306** (the default MySQL port).

5.1 Setup Steps

1. **Install MySQL Server.**

```
sudo apt-get install mysql-server
```

2. **Run the security hardening script.**

```
sudo mysql_secure_installation
```

The script will ask several questions. Recommended answers:

Prompt	Recommended Answer
Set up VALIDATE PASSWORD component?	y — then choose level 1 or 2
Remove anonymous users?	y
Disallow root login remotely?	y
Remove test database?	y
Reload privilege tables now?	y

3. Confirm MySQL is listening on port 3306.

```
sudo ss -tlnp
```

You should see:

```
LISTEN  0      151          127.0.0.1:3306          0.0.0.0:*      users:((("mysqld",pid=1389,fo
```

Notice the address is 127.0.0.1 — MySQL is currently accepting **local connections only**. You will change this in step 6.

4. Log in as root. When prompted for a password, press Enter (no password set yet).

```
sudo mysql -uroot -p
```

5. Create a remote-capable user and database.

- a. Create a new MySQL user. Replace the placeholders with your own values:

```
CREATE USER 'new-user-name'@'%' IDENTIFIED BY 'password';
```

The '%' wildcard means this user may connect from **any** IP address.

- b. Create a test database:

```
CREATE DATABASE test;
```

- c. Grant the new user full access to the test database, then flush the privileges cache:

```
GRANT ALL PRIVILEGES ON test.* TO 'new-user-name'@'%';  
FLUSH PRIVILEGES;
```

- d. Disconnect from MySQL:

Ctrl+D

- e. Log in as the new user to verify:

```
mysql -unew-user-name -p
```

- f. Check that the `test` database is visible:

```
SHOW DATABASES;
```

- g. Disconnect again with `Ctrl+D`.

6. **Enable remote connections** by changing the bind address.

- a. Open the MySQL configuration file:

```
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
```

- b. Find the `bind-address` line and change it to:

```
bind-address = 0.0.0.0
```

- c. Restart MySQL to apply the change:

```
sudo systemctl restart mysql
```

- d. Confirm MySQL is now listening on all interfaces:

```
sudo ss -tlnp
```

The relevant line should now show `0.0.0.0:3306` instead of `127.0.0.1:3306`:

```
LISTEN  0      151          0.0.0.0:3306          0.0.0.0:*      users:((("mysqld",pid=138
```

7. **Test a remote login.**

- a. (*Optional*) If you do not have a MySQL client on your host machine, coordinate with a classmate so you can use their VM as the client — or install a client using one of the options below:

- **Windows:** Download from <https://dev.mysql.com/downloads/installer/>. The smaller installer will let you choose just the command-line client.
- **macOS:** `brew install mysql-client`
- **Linux:** `sudo apt install mysql-client`

- b. Connect to your VM's MySQL server from the client machine:

```
mysql -h <VM-IP-ADDRESS> -P 3306 -u new-user-name -p
```

Replace `<VM-IP-ADDRESS>` with the IP from `ip addr show` on your VM and `new-user-name` with the user you created in step 5.

- c. Run `SHOW DATABASES;` to confirm the connection is working.

6 Summary

You have now installed and verified four core network services. Here is a quick reference for what you set up:

Service	Daemon	Default Port	Config File
HTTP	apache2	80	/etc/apache2/
FTP	vsftpd	21	/etc/vsftpd.conf
SSH	sshd	22	/etc/ssh/sshd_config
MySQL	mysqld	3306	/etc/mysql/mysql.conf.d/mysqld.cnf

Going Further

Once you are comfortable with these services, try the following to deepen your understanding:

- **Wireshark:** Capture FTP traffic and observe that your credentials appear in plaintext. Then compare with an SSH session to see the difference encryption makes.
- **Nmap:** From your host machine, run `nmap <VM-IP>` to see all four services appear as open ports — exactly what an attacker’s first scan would show.
- **fail2ban:** Install and configure **fail2ban** to automatically block IPs that repeatedly fail SSH login attempts.
- **Let’s Encrypt / Certbot:** Set up a free TLS certificate for Apache and redirect port 80 to HTTPS on port 443.